DESENVOLVIMENTO DE SISTEMAS III

MATHEUS LUIZ MASSUDA (FEJ)

ATIVIDADE 02

1 RELATÓRIO DA ATIVIDADE

O código final, iniciado na Atividade 1 e aprimorado na Atividade 2, está representado nas figuras abaixo por meio de capturas de tela do sistema em funcionamento. A aplicação desenvolvida em .NET MAUI, prioriza uma arquitetura simples e adequada à atividade. Considerando as adaptações propostas em vídeo/PDF, o padrão CRUD foi implementado de forma “parcial”, contemplando operações de inserção e leitura de dados.

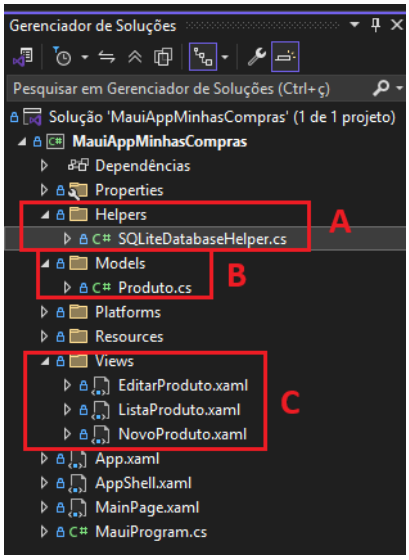
Em vez de utilizar um banco de dados relacional ou um serviço de hospedagem (host) para persistência, optou-se pelo armazenamento em arquivo de texto, com manipulação controlada por meio de leitura em modo somente leitura (ReadOnly), conforme orientação do professor Tiago. Essa abordagem possibilitou compreender os fundamentos de persistência de dados sem aumentar a escala de complexidade do sistema.

2 LINK PARA VIZUALISAÇÃO DO PROJETO (GITHUB)

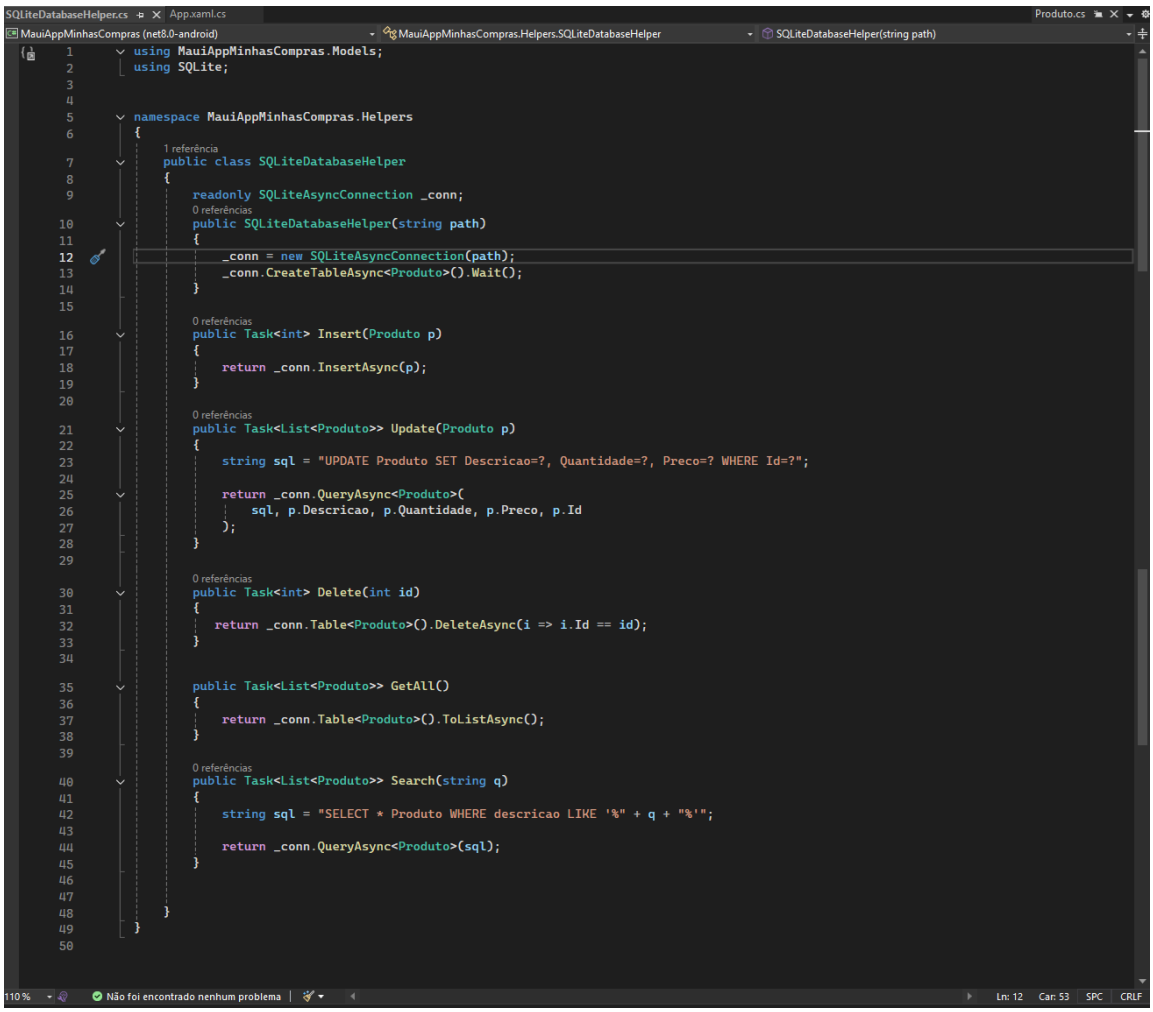
https://github.com/matheusmassuda/MauiAppMinhasCompras.git

3 CAPTURAS DE TELA NO VISUAL STUDIO (CÓDIGO)

3.1 Gerenciador de Soluções



Na seção 2.1, apresenta-se a captura de tela do gerenciador de tarefas, na qual a marcação **"A"** indica a camada de integração, responsável por conectar as páginas às funcionalidades CRUD da aplicação. Onde foram integradas as funcionalidades SQLite. Segue a figura abaixo.



```
using MauiAppMinhasCompras.Models;
using SQLite;


namespace MauiAppMinhasCompras.Helpers
{
    1 referência
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;
        0 referências
        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait();
        }

        0 referências
        public Task<int> Insert(Produto p)
        {
            return _conn.InsertAsync(p);
        }

        0 referências
        public Task<List<Produto>> Update(Produto p)
        {
            string sql = "UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?";

            return _conn.QueryAsync<Produto>(
                sql, p.Descricao, p.Quantidade, p.Preco, p.Id
            );
        }

        0 referências
        public Task<int> Delete(int id)
        {
            return _conn.Table<Produto>().DeleteAsync(i => i.Id == id);
        }


        public Task<List<Produto>> GetAll()
        {
            return _conn.Table<Produto>().ToListAsync();
        }

        0 referências
        public Task<List<Produto>> Search(string q)
        {
            string sql = "SELECT * Produto WHERE descricao LIKE '%" + q + "%'";

            return _conn.QueryAsync<Produto>(sql);
        }


    }
}
```

Ainda na seção 2.1, a marcação **“B”** identifica os modelos da aplicação, correspondentes ao trecho de código em que são definidos os tipos de dados relacionados à entidade produto. Vide captura abaixo.

```
SQLiteDatabaseHelper.cs   App.xaml.cs                                   Produto.cs
MauiAppMinhasCompras (net8.0-android)   MauiAppMinhasCompras.Models.Produto   Id
1   using SQLite;
2
3
4   namespace MauiAppMinhasCompras.Models
5   {
        10 referências
6       public class Produto
7       {
8           [PrimaryKey, AutoIncrement]
            2 referências
9           public int Id { get; set; }
            1 referência
10          public string Descricao { get; set; }
            1 referência
11          public double Quantidade { get; set; }
            1 referência
12          public double Preco {  get; set; }
13
14      }
15  }
16
```

Na mesma seção 2.1, a marcação **“C”** refere-se às “ContentPages”, isto é, às estruturas de conteúdo das páginas que serão desenvolvidas e aprimoradas nas próximas etapas do projeto. Vide capturas abaixo.

```
SQLiteDatabaseHelper.cs   App.xaml.cs                                   EditarProduto.xaml.cs
MauiAppMinhasCompras (net8.0-android)   MauiAppMinhasCompras.Views.EditarProduto   EditarProduto()
1   namespace MauiAppMinhasCompras.Views;
2
    14 referências
3   public partial class EditarProduto : ContentPage
4   {
        0 referências
5       public EditarProduto()
6       {
7           InitializeComponent();
8       }
9   }
```

```
SQLiteDatabaseHelper.cs   App.xaml.cs                                   ListaProduto.xaml.cs
1   namespace MauiAppMinhasCompras.Views;
2
3   public partial class ListaProduto : ContentPage
4   {
5       public ListaProduto()
6       {
7           InitializeComponent();
8       }
9   }
```

```
SQLiteDatabaseHelper.cs   App.xaml.cs                                   NovoProduto.xaml.cs
1   namespace MauiAppMinhasCompras.Views;
2
3   public partial class NovoProduto : ContentPage
4   {
5       public NovoProduto()
6       {
7           InitializeComponent();
8       }
9   }
```

Comentou-se a instrução “MainPage = new AppShell();” e definiu-se a página principal como “MainPage = new NavigationPage(new Views.ListaProduto());” na aplicação .NET MAUI, estabelecendo “Views.ListaProduto” como tela inicial, ainda sem alterações funcionais. Captura abaixo.

```
SQLiteDatabaseHelper.cs   App.xaml.cs                                   NovoProduto.xaml.cs
MauiAppMinhasCompras (net8.0-android)   MauiAppMinhasCompras.App   InitializeComponent()
1   namespace MauiAppMinhasCompras
2   {
        15 referências
3       public partial class App : Application
4       {
            0 referências
5           public App()
6           {
7               InitializeComponent();
8
9               //MainPage = new AppShell();
10              MainPage = new NavigationPage(new Views.ListaProduto());
11          }
12      }
13  }
14
```